



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrian Davila Perez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 1

Integrante(s): 323001315, 426057857, 323008754, 426058665,
323067267

No. de lista o

brigada: 5

Semestre: 2027-1

Fecha de entrega: Viernes 21 de Agosto

Índice

1. Introducción	2
2. Marco teórico	2
2.1. Sintaxis de Java	3
2.2. Tipos de datos	3
2.3. Variables	3
2.4. Constantes	3
2.5. Operadores y expresiones	3
2.6. Estructuras de selección	4
2.7. Estructuras de repetición	4
2.8. break y continue	4
3. Desarrollo	5
4. Resultados	5
5. Conclusiones	8

1. Introducción

- **Planteamiento del problema.** La práctica se compone de dos ejercicios desarrollados en el lenguaje Java. El primero consiste en una calculadora de consola que recibe dos números enteros y realiza sobre ellos las operaciones de suma, resta, multiplicación, división, potencia, raíz y módulo, validando los casos en los que una operación no es posible (como la división entre cero). El segundo consiste en una aplicación tipo "paint", presentada previamente por el profesor, que permite generar figuras geométricas mediante un menú de selección, asignándoles un color de relleno y permitiendo eliminar la última figura creada. Para este segundo ejercicio no se contó con el código del profesor, por lo que se recurrió a un modelo de lenguaje (LLM) para razonar y proponer una posible implementación orientada a objetos.
- **Motivación.** Ambos ejercicios permiten reforzar distintos niveles de conocimiento en Java. La calculadora aborda los fundamentos del lenguaje: declaración de variables, tipos de datos, operadores aritméticos y estructuras de control. El paint, por su parte, introduce a la Programación Orientada a Objetos, y al desarrollarse con apoyo de un LLM, permite además evaluar qué tan acertado es el razonamiento de una inteligencia artificial, fomentando el pensamiento crítico del equipo al analizar, cuestionar y comparar la propuesta obtenida.
- **Objetivos.**
 - Implementar una calculadora en Java que aplique correctamente los operadores aritméticos.
 - Recopilar, con base en la memoria y notas del equipo, las especificaciones funcionales del programa tipo paint presentado en clase.
 - Analizar críticamente la propuesta generada por el LLM, formulando preguntas y evaluando su coherencia con los principios de la POO.
 - Comparar el diseño propuesto por la IA identificando posibles mejoras o discrepancias.
 - Documentar el proceso y los hallazgos obtenidos como parte del aprendizaje colaborativo entre el equipo y la herramienta de IA.

2. Marco teórico

Para el desarrollo de la práctica se utilizan conceptos fundamentales del lenguaje Java, los cuales permiten declarar y manipular datos, realizar operaciones y controlar el flujo de ejecución de un programa. Estos elementos constituyen una base para la elaboración de programas estructurados y permiten resolver problemas mediante instrucciones organizadas [2].

2.1. Sintaxis de Java

La sintaxis establece las reglas que deben seguirse para escribir correctamente un programa. En Java, el código se organiza mediante clases, métodos, instrucciones y expresiones. Cada instrucción generalmente termina con un punto y coma, mientras que las llaves permiten delimitar bloques de código. Seguir estas reglas es necesario para que el compilador pueda interpretar correctamente el programa [3].

2.2. Tipos de datos

Los tipos de datos determinan la clase de información que puede almacenar una variable. Java cuenta con tipos de datos primitivos como `int`, `double`, `char` y `boolean`, además de tipos de referencia utilizados para trabajar con objetos. La elección del tipo depende de la información que se necesite representar en el programa [2].

2.3. Variables

Una variable es un espacio de memoria identificado mediante un nombre, cuyo contenido puede cambiar durante la ejecución del programa. Para declarar una variable en Java se indica el tipo de dato y posteriormente el nombre que tendrá. Por ejemplo, la instrucción `int numero = 10;` declara una variable de tipo entero e inicializa su valor en 10 [3].

Las variables permiten almacenar los datos que serán utilizados posteriormente en operaciones, comparaciones y procesos dentro del programa. Por esta razón, son elementos esenciales para representar la información involucrada en la solución de un problema [2].

2.4. Constantes

Una constante representa un valor que no debe modificarse después de haber sido establecido. En Java se utiliza la palabra reservada `final` para indicar que una variable tendrá un valor que no podrá ser reasignado posteriormente [2].

El uso de constantes permite mantener determinados valores sin cambios durante la ejecución del programa, lo cual resulta conveniente para datos que deben conservar el mismo valor durante todo el proceso.

2.5. Operadores y expresiones

Los operadores permiten realizar operaciones sobre valores y variables. En Java existen operadores aritméticos, de asignación, relacionales y lógicos. Los operadores aritméticos permiten realizar cálculos; los de asignación permiten asignar

valores; mientras que los relacionales y lógicos permiten realizar comparaciones y construir condiciones [3].

Una expresión puede estar formada por variables, valores y operadores, y produce un resultado que puede ser utilizado por otras instrucciones del programa. La correcta combinación de estos elementos permite realizar los cálculos y las evaluaciones necesarias para resolver un problema [2].

2.6. Estructuras de selección

Las estructuras de selección permiten elegir qué instrucciones ejecutar dependiendo de una condición. En Java, la estructura `if` permite ejecutar un bloque cuando una condición es verdadera, mientras que `if-else` permite establecer un bloque alternativo cuando dicha condición es falsa [deitel2017].

La estructura `switch` permite seleccionar entre diferentes alternativas a partir del valor de una expresión. Esta estructura puede utilizarse cuando se tienen varias opciones posibles que dependen de un mismo valor [2].

2.7. Estructuras de repetición

Las estructuras de repetición permiten ejecutar un conjunto de instrucciones varias veces. Java proporciona los ciclos `while`, `do-while` y `for`, los cuales permiten repetir instrucciones de acuerdo con una condición determinada [3].

El ciclo `while` evalúa la condición antes de ejecutar el bloque de instrucciones, por lo que es posible que el ciclo no se ejecute. El ciclo `do-while`, en cambio, ejecuta primero las instrucciones y posteriormente evalúa la condición. Por su parte, `for` permite establecer en una misma estructura la inicialización, la condición y la actualización de la variable de control [2].

2.8. `break` y `continue`

Las instrucciones `break` y `continue` permiten modificar el flujo normal de ejecución de un ciclo. La instrucción `break` termina el ciclo inmediatamente, mientras que `continue` permite omitir el resto de la iteración actual y continuar con la siguiente [3].

Estos elementos permiten controlar situaciones particulares dentro de las estructuras repetitivas y pueden utilizarse cuando se requiere terminar un proceso o pasar directamente a la siguiente iteración.

En conjunto, los conceptos anteriores permiten construir programas capaces de almacenar información, realizar operaciones, evaluar condiciones y repetir procesos. Por ello, constituyen herramientas fundamentales para plantear y desarrollar una solución mediante el lenguaje Java.

3. Desarrollo

- Inicio del programa. Se crea un objeto de la clase Scanner para permitir la lectura de datos desde el teclado.
- Declaración de variables. Se declaran las variables necesarias para almacenar los dos números ingresados y los resultados de cada operación (suma, resta, multiplicación y módulo como enteros; división, potencia y raíz como valores decimales).
- Captura de datos. Se solicita al usuario ingresar el primer número (n1) y posteriormente el segundo número (n2), almacenando cada uno conforme se van ingresando.
- Cálculo de la suma. Se obtiene sumando directamente los dos números enteros ingresados, almacenando el resultado en una variable entera.
- Cálculo de la resta. Se calcula restando el segundo número al primero, respetando el orden de los operandos.
- Cálculo de la multiplicación. Se multiplican n1 y n2, y el resultado se muestra en pantalla.
- Cálculo de la división. Se verifica si n2 es distinto de cero. Si la condición se cumple, se convierte n1 a tipo decimal y se divide entre n2, mostrando el resultado. Si n2 es igual a cero, se vuelven a solicitar los números.
- Cálculo de la potencia. Se calcula n1 elevado a n2 utilizando la función Math.pow(), y el resultado se muestra en pantalla.
- Cálculo de la raíz. Se aproximó el cálculo de la raíz utilizando también Math.pow(), elevando el primer número al inverso del segundo. Se validó que el primer número no sea negativo y que el segundo sea distinto de cero, ya que ambos casos generarían un resultado indefinido o un error de división entre cero dentro del exponente.
- Cálculo del módulo. Se verifica si n2 es distinto de cero. Si la condición se cumple, se obtiene el residuo de dividir n1 entre n2 usando el operador Si n2 es igual a cero, se muestra vuelve a solicitar el número.
- Cierre del programa. Se cierra el objeto Scanner y finaliza la ejecución.

4. Resultados

1-A continuación, se presenta la ejecución del programa `Calculadora.java` a través de la consola. Primero, una ejecución estándar ingresando los valores 10 y 5, donde todas las operaciones aritméticas se calculan de forma exitosa; en seguida, una ejecución de validación ingresando 10 y 0, el programa evalúa correctamente las restricciones matemáticas mediante las estructuras condicionales, evitando errores del sistema y notificando al usuario que la división, la raíz de índice cero y el módulo no son posibles.

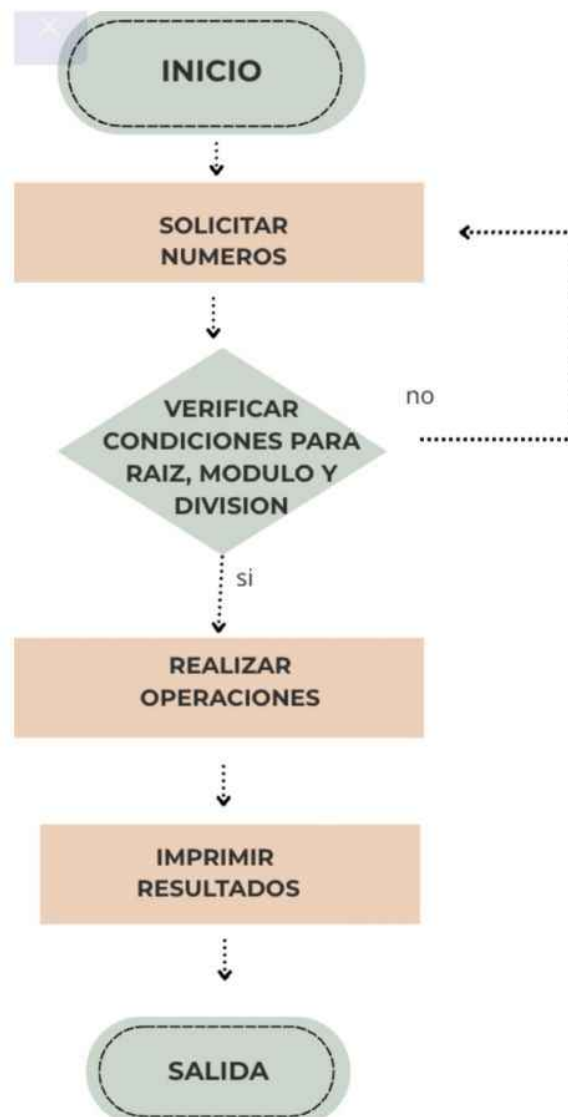


Figura 1: Enter Caption


```
PS C:\Users\acer\Desktop> cd P00
PS C:\Users\acer\Desktop\P00> ls

Directorio: C:\Users\acer\Desktop\P00

Mode                LastWriteTime         Length Name
----                -
-a----             25/08/2026   04:04 p. m.         2051 Calculadora.class
-a----             25/08/2026   04:01 p. m.         2063 Calculadora.java
-a----             21/08/2026   12:20 a. m.          425 HolaMundo.class
-a----             21/08/2026   12:03 a. m.          126 HolaMundo.java

PS C:\Users\acer\Desktop\P00> javac Calculadora.java
PS C:\Users\acer\Desktop\P00> java Calculadora
Ingrese el primer numero:
10
Ingrese el segundo numero:
5
La suma de 10 y 5 es 15
La resta de 10 y 5 es 5
La multiplicacion de 10 y 5 es 50
La division de 10 entre 5 es 2.0
La potencia de 10 elevado a 5 es 100000.0
La raiz de indice 5 de 10 es 1.5848931924611136
El modulo de 10 entre 5 es 0
PS C:\Users\acer\Desktop\P00> java Calculadora
Ingrese el primer numero:
10
Ingrese el segundo numero:
0
La suma de 10 y 0 es 10
La resta de 10 y 0 es 10
La multiplicacion de 10 y 0 es 0
La division de 10 entre 0 no es posible
La potencia de 10 elevado a 0 es 1.0
La raiz de indice 0 de 10 no es posible
El modulo de 10 entre 0 no es posible
PS C:\Users\acer\Desktop\P00> |
```

Figura 2: Ejecución del programa calculadora.

2-A la hora de preguntar sobre el código de un ejemplo de tipo paint a nuestro LLM lo que nos dio fue lo siguiente:

Estructura de Datos y Memoria (Historial)

Para evitar que los trazos sean simples píxeles estáticos y permitir la función de “Deshacer”, el sistema emplea una arquitectura orientada a objetos. Se diseñó una clase **Figura** que almacena los metadatos de cada trazo (coordenadas, color, tipo y si tiene relleno).

El lienzo mantiene un historial de estas figuras utilizando una lista dinámica. Esta es la estructura fundamental que actúa como memoria del programa:

Listing 1: Estructura de almacenamiento de figuras

```
1 // Memoria para el historial de dibujo
2 private ArrayList<Figura> listaFiguras = new ArrayList<>();
3 private Figura figuraActual = null;
```

Motor de Dibujo y Eventos del Ratón

La interacción en el lienzo se gestiona mediante un **MouseAdapter**. Cuando el usuario hace clic, se instancia un objeto **Figura** temporal. Al arrastrar

el ratón, se actualizan las coordenadas finales de esta figura y se invoca al método `repaint()`.

El renderizado real ocurre sobrescribiendo el método `paintComponent`. Este método limpia la pantalla y redibuja iterativamente todo el historial de figuras almacenado en la memoria:

Listing 2: Motor de renderizado gráfico

```
1 @Override
2 protected void paintComponent(Graphics g) {
3     super.paintComponent(g); // Limpia el lienzo
4     Graphics2D g2d = (Graphics2D) g;
5
6     // Redibuja el historial completo
7     for (Figura f : listaFiguras) {
8         f.dibujar(g2d);
9     }
10 }
```

Lógica de los Controles de Edición

Gracias a la implementación del `ArrayList`, las funciones de edición se reducen a manipular esta estructura de datos.

Para la función **Deshacer**, el sistema verifica si el historial contiene elementos. Si es así, elimina el último objeto de la lista y solicita un repintado de la interfaz, lo que hace desaparecer el último trazo visualmente.

Listing 3: Lógica de la función `Deshacer`

```
1 public void deshacer() {
2     if (!listaFiguras.isEmpty()) {
3         listaFiguras.remove(listaFiguras.size() - 1);
4         repaint();
5     }
6 }
```

Para la función **Borrar Todo**, el programa simplemente ejecuta `listaFiguras.clear()` y vuelve a pintar el lienzo en blanco.

5. Conclusiones

Con base en los resultados se puede decir que los objetivos principales de la práctica sí se cumplieron, la calculadora realizó correctamente las operaciones solicitadas y validó casos como la división o el módulo entre cero, además se mantuvo una estructura sencilla utilizando variables, operadores, condiciones, `Scanner` para recibir datos y métodos de la clase `Math`

cuando eran necesarios, por lo que se resolvió el problema con las herramientas básicas de Java sin agregar procesos más complicados de los que pedía el ejercicio

En la parte de la aplicación tipo Paint, la respuesta de la IA propuso una solución orientada a objetos usando clases, objetos, un ArrayList para guardar las figuras y eventos del ratón para controlar el dibujo, pero también mostró que una IA puede proponer una solución más compleja o diferente a la esperada cuando no recibe instrucciones suficientemente específicas, por lo que el resultado confirma la importancia de indicar claramente qué se necesita, el nivel con el que se está trabajando y después revisar si la propuesta realmente corresponde con el problema planteado

Referencias

- [1] H. Schildt, *Java: The Complete Reference*, 11th ed. New York, NY, USA: McGraw-Hill Education, 2018.
- [2] P. Deitel and H. Deitel, *Java: How to Program*, 11th ed. Boston, MA, USA: Pearson, 2017.
- [3] M. A. Nakayama C. y J. A. Solano Gálvez, *Manual de prácticas del Laboratorio de Programación Orientada a Objetos*, cód. MADO-22, versión 01, Facultad de Ingeniería, UNAM, México, 20 de enero de 2017.